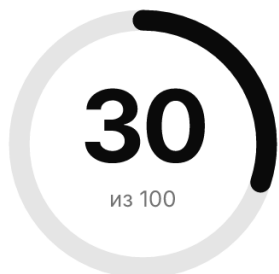


homyakin/jlint

Java

Оценено 28 сентября 2026 г.

Данных собрано 53%



БАЛЛ ЗДОРОВЬЯ

Слабое состояние: многое стоит поправить.

ЧТО ЭТО · О ПРОЕКТЕ

Jlint — инструмент для проверки и форматирования Java-кода, вдохновлённый ktlint; текущая версия предоставляет базовую инфраструктуру без реализованных правил линтинга. Репозиторий включает ядро, CLI, текстовый отчётчик и интеграции с Maven и Gradle, а также примеры проектов и тесты.

- линтер и форматировщик Java 17+
- CLI-команды проверки и форматирования
- плагин Maven с целями check и format
- интеграция с Gradle — базовая библиотека
- текстовый формат отчётов

● Безопасность



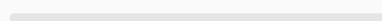
● Код

61



● Активность

0



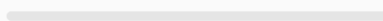
● Документация

49



● CI/CD

0



● Задачи



Категории и метрики

● **Безопасность** вес 20%

Critical-находки AppSec

Нет данных SourceCraft AppSec

High-находки AppSec

Нет данных SourceCraft AppSec

Medium-находки AppSec

Нет данных SourceCraft AppSec

Секреты в репозитории (AppSec)

Нет данных SourceCraft AppSec

● **Код** вес 20%

61

Ревью кода моделью

Код разделён на понятные модули, а показанные тесты проверяют реальные сценарии, особенно для правила отступов. Однако некоторые важные ошибки превращаются в успешный результат или тихий переход к значениям по умолчанию; крупный `IndentationRule` также концентрирует много логики в одном классе.

61

● Активность вес 15%

0

Коммитов за 90 дней

Коммитов за 90 дней: 0

0

Активных авторов

Активных авторов за 90 дней: 0

0

Свежесть последнего коммита

Последний коммит: 327 дн. назад

0

Bus factor

Не удалось посчитать долю кода топ-автора

—

Релизы и теги

Релизов и тегов нет

0

Поток pull request

Pull request не используются

0

● Документация вес 15%

49

Документация по рубрике модели

README хорошо разбит на разделы и содержит примеры CLI и сборки, но не даёт полноценной инструкции по установке и подробной справки по API. Документация выглядит устаревшей: README заявляет об отсутствии linting rules и поддержки .editorconfig, хотя в списке файлов есть соответствующие реализации; также примеры плагинов стоит сверить с реальными пакетами и модулями.

49

● CI/CD вес 15%

0

Конфиг CI

Конфига CI нет

0

Тесты в пайплайне

Конфига CI нет — проверять нечего

—

Линтер в пайплайне

Конфига CI нет — проверять нечего

—

Проверка pull request

Конфига CI нет — проверять нечего

—

Успешные прогоны CI

Прогоны CI видны только участникам — в публичный балл не входят

—

● Задачи вес 15%

Закрытие задач

Трекером задач не пользуются

Заброшенные задачи

Трекером задач не пользуются

Скорость реакции на задачи

Трекером задач не пользуются

Что сделать

1 Настройте CI +23

Сейчас: Конфига CI нет

Создайте `.sourcecraft/ci.yaml`: запуск на `push` и `pull_request`, шаги — `./gradlew check` или `mvn verify`. Pulse может подготовить этот файл в `pull request`.

Усилия: день

2 Зафиксируйте активность свежим коммитом +5

Сейчас: Последний коммит: 327 дн. назад

Проект давно не обновлялся. Если он жив — обновите зависимости и выпустите версию; если нет — отметьте это в `README`.

Усилия: час

3 Выпустите релиз и поставьте тег версии +4

Сейчас: Релизов и тегов нет

Поставьте тег версии (например, `v1.0.0`) и опубликуйте релиз с описанием изменений — и заведите `CHANGELOG`.

Усилия: час

4 **Наращивайте темп разработки** **+4**

Сейчас: Коммитов за 90 дней: 0

Коммитьте чаще и мельче: одна законченная правка — один коммит. Это и ритм проекта, и удобное ревью.

Усилия: день

5 **Проводите изменения через pull request** **+2**

Сейчас: Pull request не используются

Вносите изменения через pull request даже в одиночку: история решений и место для проверок CI.

Усилия: день

6 **Привлекайте ещё контрибьюторов** **+3**

Сейчас: Активных авторов за 90 дней: 0

Заведите задачи с меткой для новичков и опишите в CONTRIBUTING, как начать: так проще привлечь соавторов.

Усилия: неделя и больше

Каких данных нет

- AppSec SourceCraft отдаёт результаты только участникам репозитория — в публичный балл безопасность не входит